

LibSmart

VS Code plugin for writing modern code

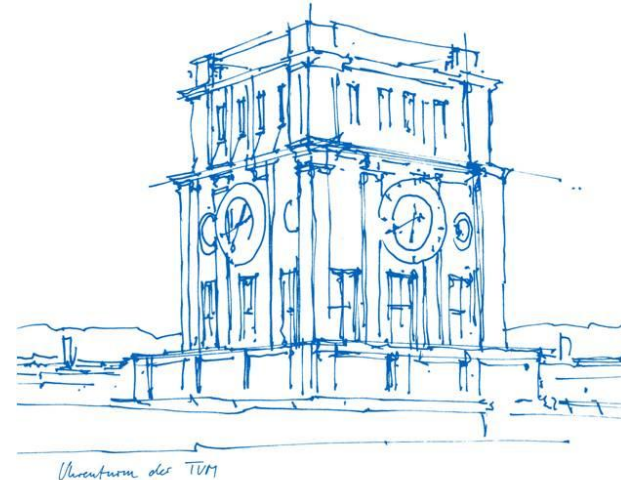
Priyam Turakhia, Julian Pins, Frytz Martinek, Alena Bobovich

Bachelor practical course Next Gen Programming

Technische Universität München

Group: McMurdo

Supervisor: Mohammed Kasra Habib, Tobias Eisenreich



Introduction

Problem & Solution



Problem & Solution



Problem:

constant updates of libraries
→ code modernization is
tedious and error prone



Our solution:

A smart VS-Code plugin
for python developers →
better library usage.



Key features:

Context-aware assistance:
detect deprecated features,
suggest modern alternatives

Introduction

Development Process



Development Process

Rhythm: Sprints with clear goals

Ceremonies: Sprint reviews & retros: demos, blockers, re-prioritization



Development Process

Rhythm: Sprints with clear goals

Ceremonies: Sprint reviews & retros: demos, blockers, re-prioritization



Objective: Research, Data collection and backend prototype

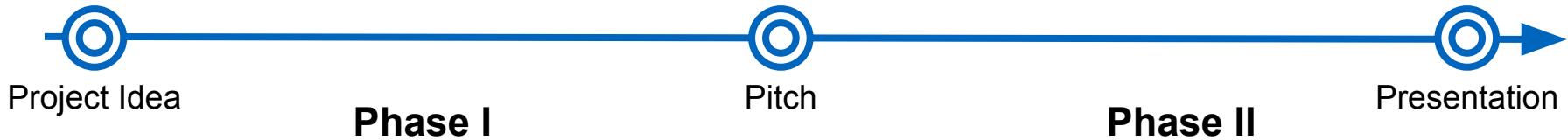
Tasks:

- Backend & RAG: Alena, Priyam
- Data scraping & eval script: Frytz, Julian

Development Process

Rhythm: Sprints with clear goals

Ceremonies: Sprint reviews & retros: demos, blockers, re-prioritization



Objective: Research, Data collection and backend prototype

Tasks:

- Backend & RAG: Alena, Priyam
- Data scraping & eval script: Frytz, Julian

Objective: Fine-tune & evaluate models and VS Code extension

Tasks:

- Fine-tuning & eval: Entire team
- Model integration (GGUF, server): Priyam
- Front-end (VS Code): Julian

Method

Dataset & Data Pipeline



Training Dataset & Data Pipeline

Composition

100% gemini generated code snippets

Statistics

Deprecated numpy features: 40

Code samples per feature: 5-8

Total validated samples: 248

Schema

input	<code>scalar_val = np.asscalar(np.array([42]))</code>
package	<code>numpy</code>
version	<code>1.16.0</code>
context	"The function <code>numpy.asscalar</code> is deprecated since NumPy 1.16.0. Use <code>numpy.array.item</code> instead."
functionality	"Convert an array of size 1 to its corresponding Python scalar."
output	<code>scalar_val = np.array([42]).item()</code>

RAG Pipeline:

Step 1: Chunk Extraction

Step 2: Embedding

Step 3: Vector Store (ChromaDB)

Method

Model Training



Model training

Environment



Platform: Google Colab

Bottlenecks:

- 16 GB VRAM limitation
- Frequently ran out of RAM and compute time

Method



Optimization techniques:

- Parameter-efficient Fine-Tuning
- LoRA's approach
- 4-bit quantization (QLoRA)

Pain Points



Fine-tuning Challenges:

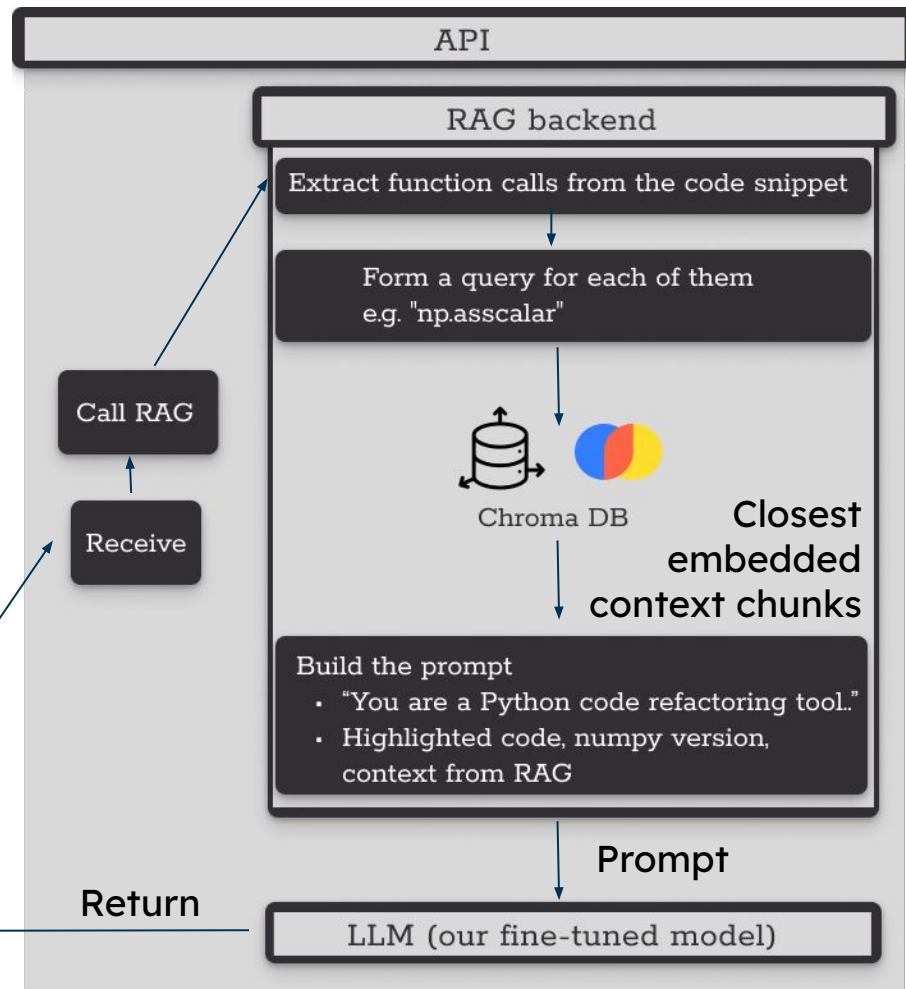
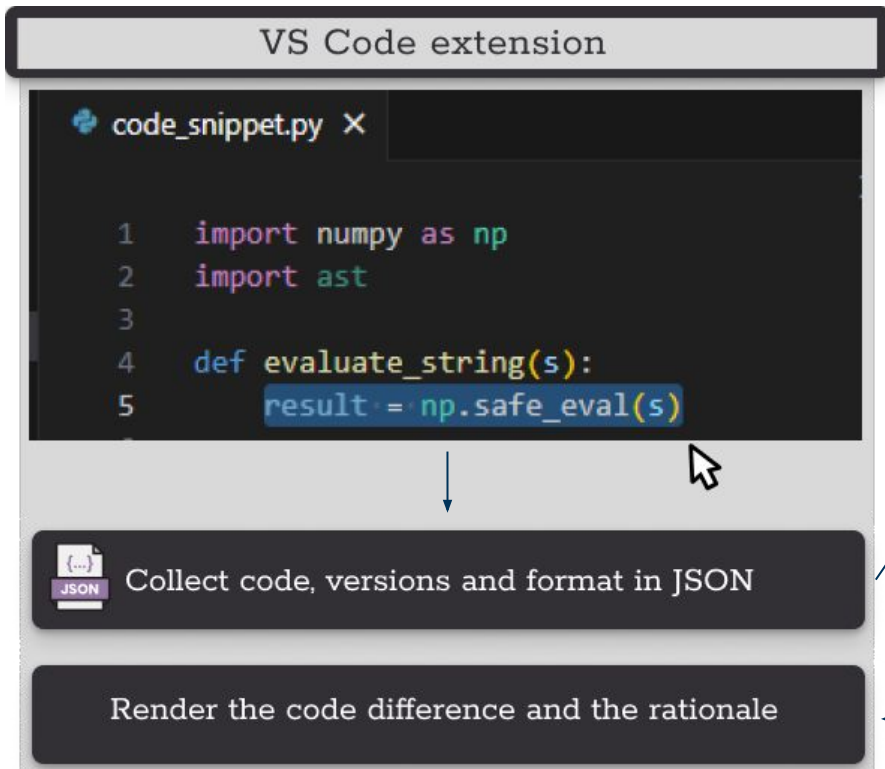
- Large model sizes
- Dependency conflicts
- Finding a proper prompt
- suitable hyperparameters
- Empty outputs

Method

System Architecture



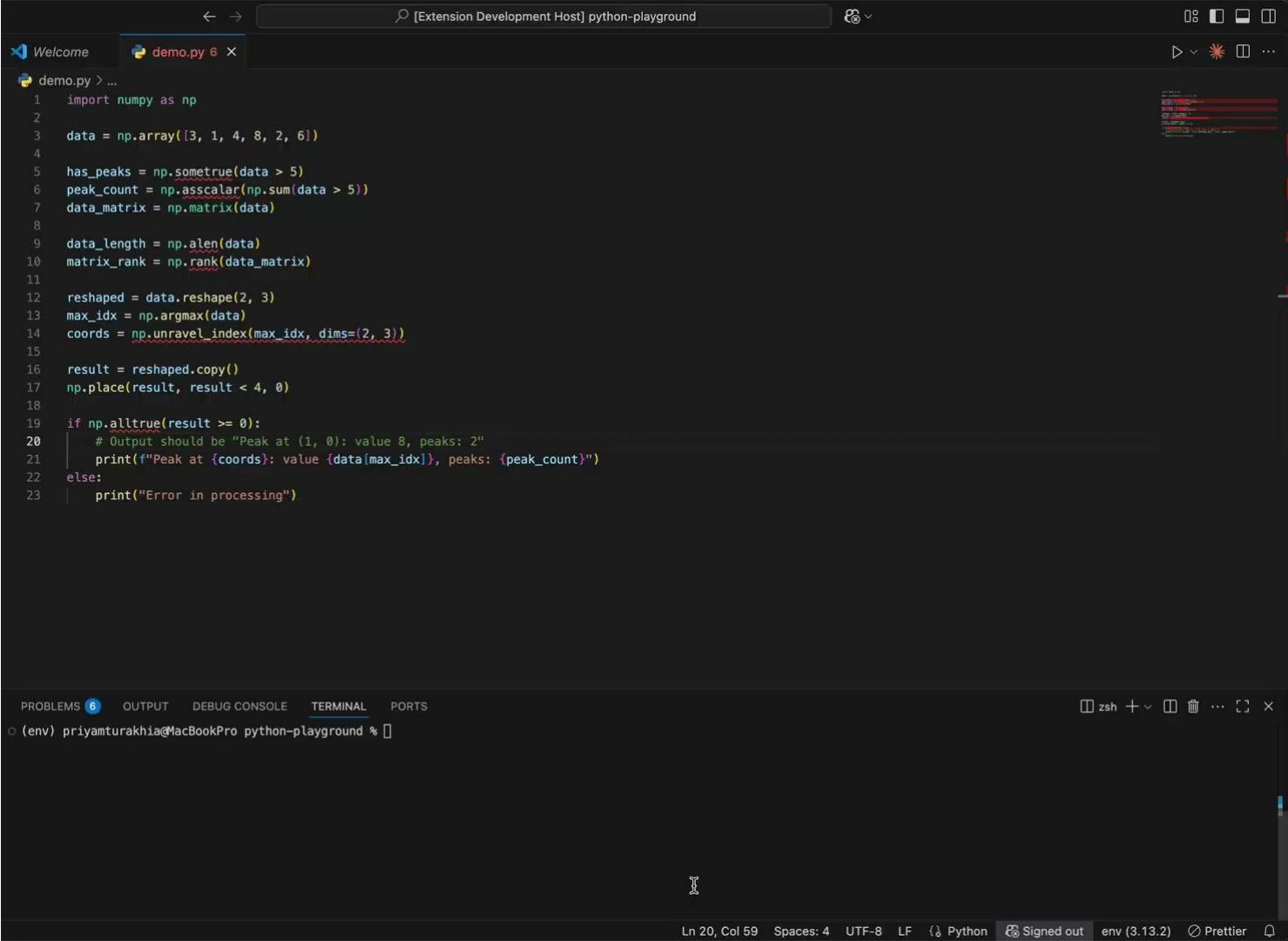
System architecture



Results

Demo

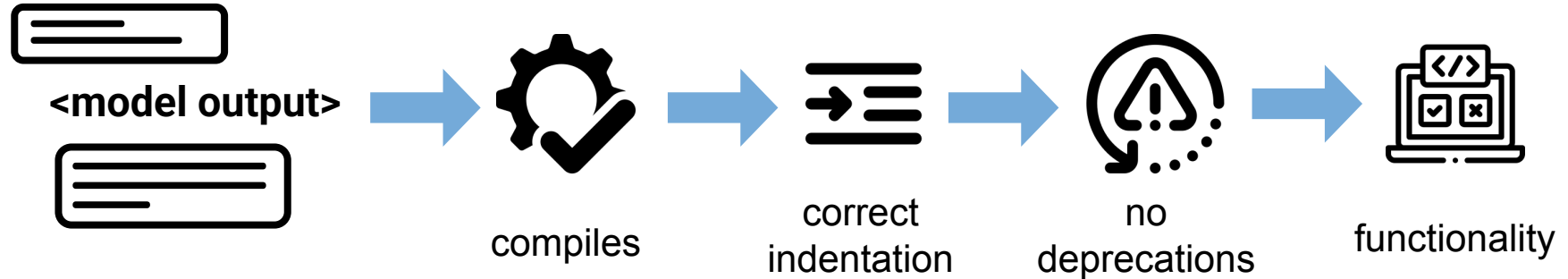




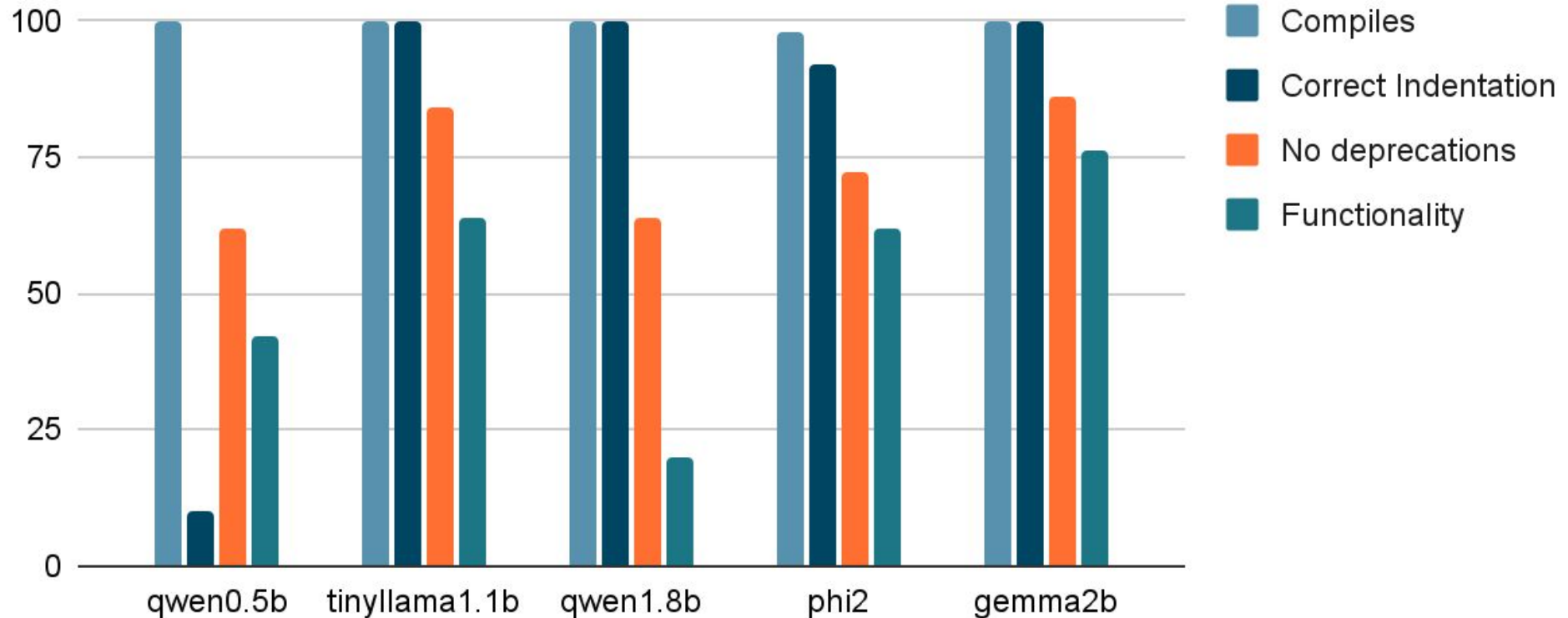
Results Evaluation



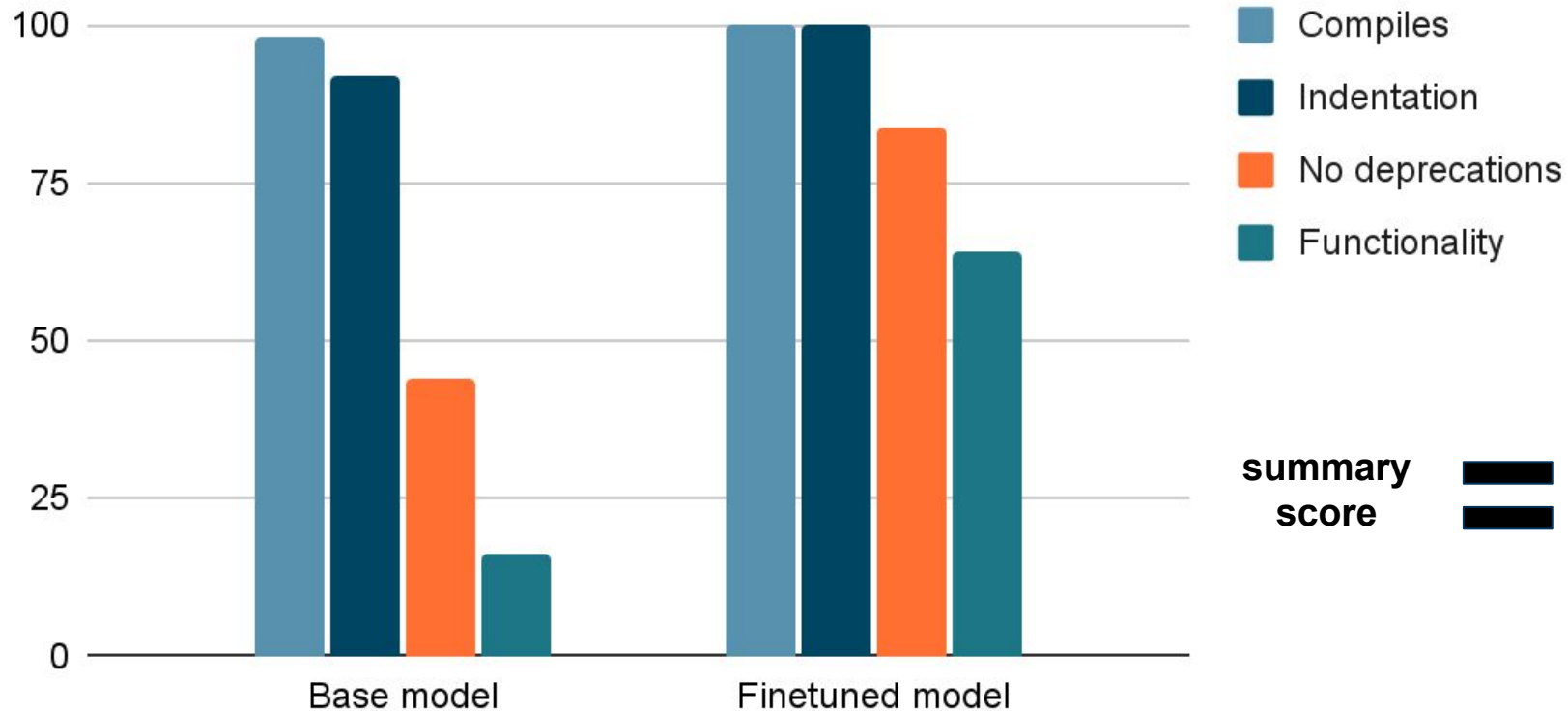
Evaluation Metrics



Model comparison : post fine-tuning



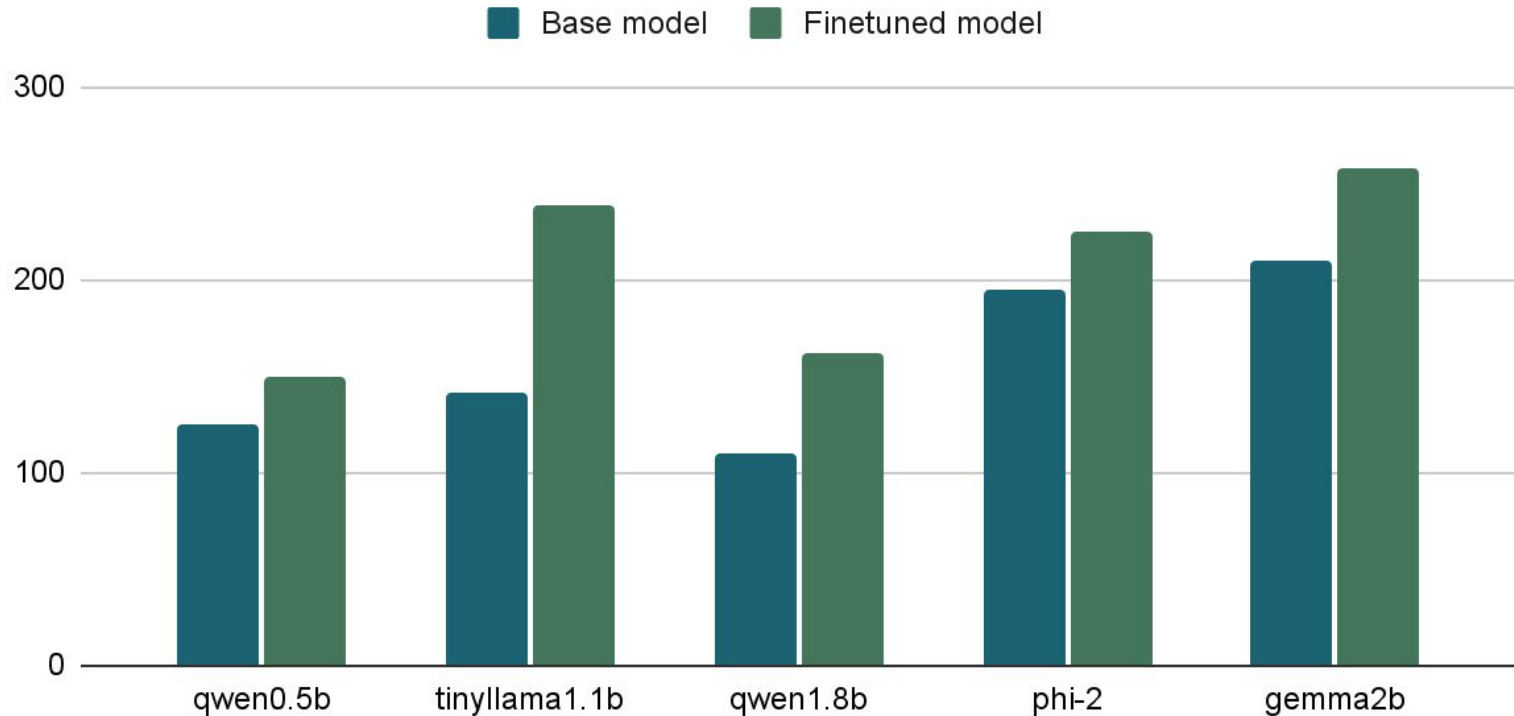
TinyLlama1.1b: Base vs. Finetuned model



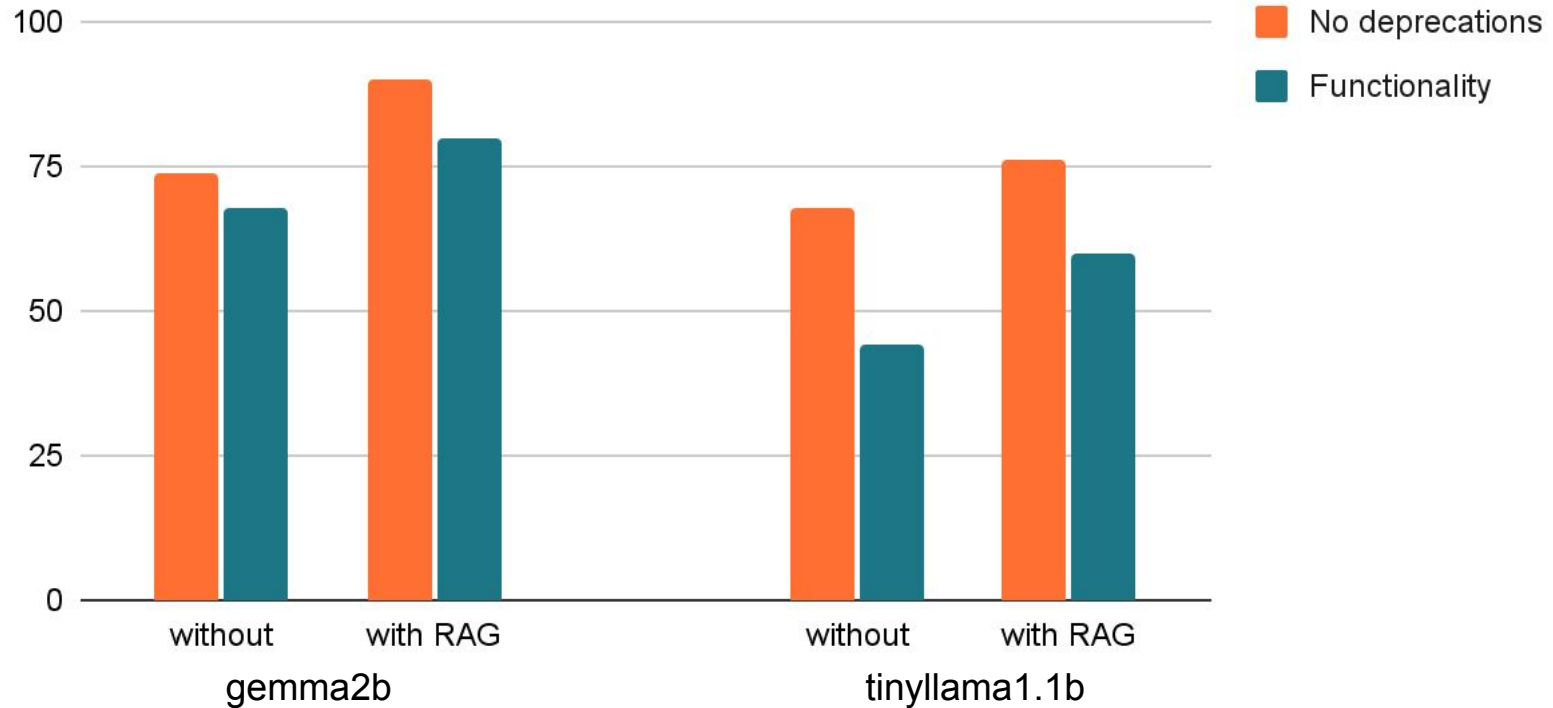
summary
score



Model comparison summary



RAG Improvement



Discussion

Limitations & Lessons Learned



Limitations & Lessons Learned

Data Scraping

✗ <10 % of scraped Git commits were usable

✗ Regex/ LangChain
~50 % chunks unusable

Backend - RAG

✗ Regex missed patterns like `np.random.*`

✗ unpredictable output

Fine-tuning

✗ T4 VRAM limit on Colab during fine-tune

✗ High LR (5×10^{-5}) made the model produce only `<eos>`

Evaluation

✗ struggling with suitable metrics & testing

Limitations & Lessons Learned

Data Scraping

✗ <10 % of scraped Git commits were usable

✗ Regex/ LangChain
~50 % chunks unusable

✓ Switched to manual
symbol list + Gemini
generation

✓ Simple HTML→
Markdown pass

Backend - RAG

✗ Regex missed patterns
like `np.random.*`

✗ unpredictable output

✓ Replaced regex with
AST-based parsing

✓ Strict prompt template

Fine-tuning

✗ T4 VRAM limit on
Colab during fine-tune

✗ High LR (5×10^{-5})
made the model produce
only `<eos>`

✓ Used 4-bit LoRA
quantisation

✓ max 2B parameter
models

Evaluation

✗ struggling with
suitable metrics & testing

✓ only evaluate using
instruction-tuned models

+ many more

Conclusion



Conclusion



What we built:

- NumPy code refactoring assistant
- Integrated the best model into a VS Code extension



What we learned:

- fine-tuning lifecycle
- RAG
- VS Code extension



Outlook / Further development:

- Extend to different libraries
- More reliable ChromaDB
- Bayesian optimization

Thank you for your attention

